



포팅메뉴얼

0. 목차

0. 목차

1. 개발 환경

Backend

Frontend

CI/CD

Cooperation Tools

2. 환경변수 정보

Backend

Frontend

3. 배포 관련 설정

3-1. EC2 설정

— 가상 메모리 할당

3-2 Nginx 설정

— 방화벽 설정

— 포트 개방

3-3. SSL 설정

— SSL 인증서 발급

— Nginx default.conf 파일

3-4. 도커 설정

— 사전 설정: 업데이트 및 HTTP 패키지 설치

— 사전 설정: GPG 키 및 저장소 추가

— 도커 설치

3-5. 젠킨스 설정

— 환경 설정 변경

— 젠킨스 plugin 설치

— 젠킨스 Credentials

3-6. Dockerfile 작성

— Spring Dockerfile

— React Dockerfile

3-7. Jenkinsfile 작성

3-8. ELK 스택 설정 - docker-compose.yml

4. 외부 서비스 가입 및 활용에 필요한 정보

4-1. 네이버 로그인 API

4-2. 구글 지도 API

4-3. 네이버 지도 API

4-4. 토스페이먼츠 API

5. 시연 시나리오

5-1. 로그인

5-2. 동승 구인 글 작성 (파티 생성)

5-3. 채팅

5-4. 파티 목록 조회

5-5. 자동 매칭

5-6. 포인트 충전

5-7. 정산하기

5-8. 동승자 평가 : 오디 당도

5-9. 신고하기

5-10. 파티 신청

1. 개발 환경

Backend

- JVM OpenJDK 21
- Spring Boot 3.2.4
- Gradle 8.5
- Spring Boot JPA
 - Querydsl
- Spring Security
- OAuth

Frontend

- React ^18.2.0
- react-query ^5.32.0
- typescript ^5.2.2
- react-router-dom
- axios ^1.6.8
- googlemaps/react-wrapper ^1.1.35
- stomp ^7.0.0

- STOMP
- Lombok, jjwt 0.12.3
- Swagger 2.3.0
- ELK
 - elasticsearch 8.6.2
 - kibana 8.6.2
 - logstash 8.6.2
- DB
 - MariaDB
 - S3

- zustand ^4.5.2
- tailwindcss ^3.4.3
- daisyui ^4.10.2
- framer-motion ^11.1.9
- eslint ^8.57.0
- prettier ^3.2.5
- tosspayments/payment-sdk ^1.7.1
- react-toastify ^10.0.5
- Visual Studio Code
 - Auto Rename Tag
 - Highlight Matching Tag
 - HTML CSS Support
 - indent-rainbow
 - Error Lens
 - Prettier - Code formatter
 - Code Spell Checker
 - Gitmoji
 - Import Cost
 - PostCSS Language Support
 - Simple React Snippets
 - Tailwind CSS IntelliSense

CI/CD

- AWS EC2 Ubuntu 20.04 LTS
- Docker
- nginx
- Jenkins

Cooperation Tools

- Gitlab
- Notion
- JIRA
- Mattermost
- Discord

2. 환경변수 정보

Backend

application.yml

```
spring:
  profiles:
    active: local
  group:
    local: db-local
    server: db-server
  include:
    - db
    - security
datasource:
  driver-class-name: ${DATABASE_DRIVER}
  url: ${DATABASE_URL}
  username: ${DATABASE_USERNAME}
  password: ${DATABASE_PASSWORD}
data:
  redis:
    host: ${REDIS_HOST}
    port: ${REDIS_PORT}
#    password: ${REDIS_PASSWORD}
jpa:
  hibernate:
    ddl-auto: ${JPA_HIBERNATE_DDL_AUTO}
  open-in-view: false
  dialect: org.hibernate.spatial.dialect.mysql.MySQL8SpatialDialect
```

```

security:
  oauth2:
    client:
      registration:
        naver:
          client-name: naver
          client-id: ${NAVER_CLIENT_ID}
          client-secret: ${NAVER_SECRET}
          redirect-uri: ${BACK_BASE_URL}/login/oauth2/code/naver
          authorization-grant-type: authorization_code
          scope: nickname, name, email, gender, birthday, profile_image, birthyear

      provider:
        naver:
          authorization_uri: https://nid.naver.com/oauth2.0/authorize
          token-uri: https://nid.naver.com/oauth2.0/token
          user-info-uri: https://openapi.naver.com/v1/nid/me
          userNameAttribute: response

elasticsearch:
  username: ${ELASTIC_USERNAME}
  password: ${ELASTIC_PASSWORD}
  uris: ${ELASTIC_URL}:${ELASTIC_PORT}

mail:
  host: smtp.gmail.com
  port: 587
  username: ${MAIL_USERNAME}
  password: ${MAIL_PASSWORD}
  properties:
    mail:
      smtp:
        auth: true
        timeout: 5000
        starttls:
          enable: true

springdoc:
  api-docs:
    path: /api/api-docs
  swagger-ui:
    doc-expansion: none
    tags-sorter: alpha
    operationsSorter: method
    disable-swagger-default-url: true
    display-request-duration: true
  info:
    title: 🍇 OD! API Server
    description: <b>오디</b>까지 가세유~? 🚗
    version: 0.0.1
  path: /api/

jwt:
  secret-key: ${JWT_SECRET_KEY}

payment:
  toss:
    test_client_api_key: ${TOSS_CLIENT_API_KEY}
    test_secret_api_key: ${TOSS_SECRET_API_KEY}

cloud:
  aws:
    s3:
      bucket: ${S3_BUCKET}
      folder:
        profile: ${S3_FOLDER_PROFILE_IMAGE}
        receipt: ${S3_FOLDER_RECEIPT}
        report: ${S3_FOLDER_REPORT}
      credentials:
        access-key: ${S3_ACCESS_KEY}
        secret-key: ${S3_SECRET_KEY}
      region:

```

```

        static: ap-northeast-2
        auto: false
    stack:
        auto: false

# Websocket 로그
logging:
    level:
        org.springframework.web.socket: DEBUG
        org.springframework.messaging: DEBUG

```

application-db.yml

- .gitignore
- Jenkins secret file 관리

```

spring:
    config:
        activate:
            on-profile: db-server

#mariaDB
DATABASE_DRIVER: org.mariadb.jdbc.Driver
DATABASE_URL: jdbc:mariadb://stg-yswa-kr-practice-db-master.mariadb.database.azure.com:3306/S10P31D201?serverTimezone=UTC
DATABASE_USERNAME: S10P31D201
DATABASE_PASSWORD: seonggoljingo16dupum
JPA_HIBERNATE_DDL_AUTO: update
---
# elastic
logging:
    level:
        org.springframework.elasticsearch.client.WIRE: TRACE

ELASTIC_USERNAME: elastic
ELASTIC_PASSWORD: QGTEFN*gurn-2eqoV0IR
ELASTIC_URL: 43.203.199.64
ELASTIC_PORT: 9200

# redis
# server
REDIS_HOST: 43.203.199.64
REDIS_PORT: 6379
REDIS_PASSWORD: homegravity201

# S3
S3_BUCKET: our-destination
S3_ACCESS_KEY: AKIAROKDTZBM3VBI5MTG
S3_SECRET_KEY: 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
S3_FOLDER_PROFILE_IMAGE: profile/
S3_FOLDER_RECEIPT: receipt/
S3_FOLDER_REPORT: report/

```

application-security.yml

- .gitignore
- Jenkins secret file 관리

```

WHITE_LIST: /oauth2/**, /login, /reissue, /api/swagger-ui/**, /api/api-docs/**, /matching/**, /ws-stomp/**, /pub/**, /sub/**

#server
FRONT_BASE_URL: https://our-destination.com
BACK_BASE_URL: https://our-destination.com

# jwt
JWT_SECRET_KEY: odi201homegravityyourdestinationisthebestprojectintheworldhyunkimbaekimleeyeo

# oauth
NAVER_CLIENT_ID: LMSBPeOxTC0AQN4Jgt0W
NAVER_SECRET: TDBkC89lZY

```



```

# map
NAVER_MAP_API_KEY_ID: g9qk3ljx3z
NAVER_MAP_API_KEY: YHtsAppjLCT9gvYgKLne4spxVY4wmre0kIi5ZNYX

# toss pay
TOSS_CLIENT_API_KEY: test_ck_EP59LybZ8BKLPABxZy4Z36GYo7pR
TOSS_SECRET_API_KEY: test_sk_50WRapdA8dbqxXpBGygAVo1zEqZK

# Gmail SMTP
MAIL_USERNAME: hamahama678900
MAIL_PASSWORD: vvgpucsqqglyoqaln
MANAGER_EMAIL: dev.soominkim@gmail.com, elegant518@gmail.com, yongarist@gmail.com, sgbae777@gmail.com, shonee99@naver.co

# Security role auth
PATH:
  /api/parties/(\d+)/(\d+), /api/parties/(\d+)/arriving, /api/parties/(\d+)/settlement, /api/parties/(\d+)/success, /api/

pathallow:
  pathSecurity:
    /api/parties/(\d+):
      POST:
        exclude: ORGANIZER
      DELETE:
        include: REQUESTER, PARTICIPANT
    /api/parties/(\d+)/(\d+):
      PUT:
        include: ORGANIZER
      DELETE:
        include: ORGANIZER
    /api/parties/(\d+)/success:
      POST:
        include: ORGANIZER

```

Frontend

.env.production (배포환경)

- .gitignore
- Jenkins secret file 관리

```

VITE_BASE_URL=https://our-destination.com
VITE_NAVER_CLIENT_ID=LMSBPe0xTC0AQN4Jgt0W
VITE_NAVER_REDIRECT_URI=https://our-destination.com/auth
VITE_KAKAO_MAP_API_KEY=
VITE_NAVER_MAP_API_KEY=YHtsAppjLCT9gvYgKLne4spxVY4wmre0kIi5ZNYX
VITE_NAVER_MAP_API_KEY_ID=g9qk3ljx3z
VITE_GOOGLE_MAP_API_KEY=AIzaSyDy1IP7S1HVn9VdXixSCIowbdQxe9DuLaw
VITE_GOOGLE_MAP_API_KEY_ID=f084d3f546f2f596
VITE_TOSS_PAYMENTS_CLIENT_KEY=test_ck_EP59LybZ8BKLPABxZy4Z36GYo7pR
VITE_PAY_URL=https://our-destination.com
VITE_SOCK_URL=wss://our-destination.com/ws-stomp
VITE_MATCHING_SOCK_URL=wss://our-destination.com/matching

```

3. 배포 관련 설정

3-1. EC2 설정

```

sudo timedatectl set-timezone Asia/Seoul

sudo vi /etc/apt/sources.list
:s/s/ap-northeast-2.ec2.archive.ubuntu.com/mirror.kakao.com/

sudo apt update

sudo apt update
sudo apt upgrade

```

— 가상 메모리 할당

```
sudo dd if=/dev/zero of=/swapfile bs=128M count=64

sudo chmod 600 /swapfile

sudo mkswap /swapfile

sudo swapon /swapfile

sudo swapon -s

sudo vi /etc/fstab

/swapfile swap swap defaults 0 0
```

3-2 Nginx 설정

— 방화벽 설정

```
sudo ufw status => 포트 확인

sudo ufw allow {포트번호}
```

— 포트 개방

포트번호	사용처
22	ssh
80	http
443	https
8989	gerrit
8080	spring
8000	jenkins
3000	react
6379	redis
9200	elasticsearch
5601	kibana

3-3. SSL 설정

— SSL 인증서 발급

```
sudo apt-get install letsencrypt -y

sudo apt install certbot python3-certbot-nginx

sudo certbot --nginx
```

— Nginx default.conf 파일

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    root /var/www/html;
    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }
}

server {
    listen 443 ssl http2 default_server;
    listen [::]:443 ssl http2 default_server;
```

```

server_name our-destination.com;
ssl_certificate /etc/letsencrypt/live/our-destination.com/fullchain.pem; # managed by Certbot
ssl_certificate_key /etc/letsencrypt/live/our-destination.com/privkey.pem; # managed by Certbot

include /etc/letsencrypt/options-ssl-nginx.conf;
ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

location /oauth2/ {
    proxy_pass http://localhost:8080/oauth2/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /login {
    proxy_pass http://localhost:8080/login;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /reissue {
    proxy_pass http://localhost:8080/reissue;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /api/swagger-ui/ {
    proxy_pass http://localhost:8080/api/swagger-ui/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /api/api-docs/ {
    proxy_pass http://localhost:8080/api/api-docs/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /matching/ {
    proxy_pass http://localhost:8080/matching/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /ws-stomp/ {
    proxy_pass http://localhost:8080/ws-stomp/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /pub/ {
    proxy_pass http://localhost:8080/pub/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /sub/ {
    proxy_pass http://localhost:8080/sub/;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}

location /api/ {
    proxy_pass http://localhost:8080/api/;

```

```

        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
        client_max_body_size 1G;
    }

    location /service-worker.js {
        root /var/www/html;
        add_header Content-Type application/javascript;
    }

    location / {
        proxy_pass http://localhost:3000;
        client_max_body_size 1G;
    }

    location /jenkins {
        proxy_pass http://localhost:8000;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
    }

    root /var/www/html;
    index index.html index.htm index.nginx.debian.html;
}

server {
    listen 80 ;
    listen [::]:80 ;
    server_name our-destination.com;

    return 301 https://$host$request_uri;
}

```

3-4. 도커 설정

— 사전 설정: 업데이트 및 HTTP 패키지 설치

```

sudo apt-get install -y ca-certificates \
curl \
software-properties-common \
apt-transport-https \
gnupg \
lsb-release

```

— 사전 설정: GPG 키 및 저장소 추가

```

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu
sudo apt-get update

```

— 도커 설치

```

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

// 설치 확인
docker -v
docker compose version

```

3-5. 젠킨스 설정

— 환경 설정 변경

```
# jenkins-data 폴더로 이동
cd /home/ubuntu/jenkins-data

# update center에 필요한 CA파일을 다운로드
mkdir update-center-rootCAs
wget https://cdn.jsdelivr.net/gh/lework/jenkins-update-center/rootCA/update-center.crt -O ./update-center-rootCAs/update-center.crt
```

— 젠킨스 plugin 설치

- GitLab 1.8.0
- Generic Webhook Trigger 2.0.1
- GitLab API 5.3.0
- Pipeline: Stage View 2.34
- Docker 1.6
- Docker Commons
- Docker Pipeline
- Docker API 3.3.4
- Mattermost Notification 3.1.3
- ssh agent
 - ssh key 등록

— 젠킨스 Credentials

- User 등록
- Docker hub 계정 등록
- 환경변수 Secret file 등록

3-6. Dockerfile 작성

— Spring Dockerfile

- path: /Backend

```
FROM gradle:8.5-jdk21 as builder
WORKDIR /app

COPY build.gradle settings.gradle /app/
COPY src /app/src

RUN gradle build -x test --parallel --continue

FROM openjdk:21-jdk-slim

COPY --from=builder /app/build/libs/*.jar /app/app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/app.jar"]
```

— React Dockerfile

- path: /Frontend

```
FROM node:20.12.2 as builder
WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build || exit 1
```

```

FROM node:20.12.2
WORKDIR /app

COPY --from=builder /app/dist /app/dist

RUN npm install -g serve

EXPOSE 3000

ENTRYPOINT ["serve", "-l", "3000", "-s", "dist"]

```

3-7. Jenkinsfile 작성

```

pipeline {
    agent any

    stages {
        stage("CI/CD start") {
            steps {
                script {
                    def Author_ID = sh(script: "git show -s --pretty=%an", returnStdout: true).trim()
                    def Author_Name = sh(script: "git show -s --pretty=%ae", returnStdout: true).trim()

                    mattermostSend (
                        color: '#D0E0E3',
                        icon: "https://jenkins.io/images/logos/jenkins/jenkins.png",
                        message: "배포 출발 합니다 🚀 \n${env.JOB_NAME} #${env.BUILD_NUMBER} by ${Author_ID}(${Author_Name})"
                    )
                }
            }
        }

        stage("Clone Repository") {
            steps {
                echo '클론 시작'
                git branch: 'release', credentialsId: 'odi', url: 'https://lab.ssafy.com/s10-final/S10P31D201.git'
                echo '클론 끝'
            }
        }

        stage("secret.yml download") {
            steps {
                withCredentials([file(credentialsId: 'secret-db', variable: 'dbConfigFile')]) {
                    script {
                        sh 'cp -rf $dbConfigFile ./Backend/src/main/resources/application-db.yml'
                    }
                }

                withCredentials([file(credentialsId: 'secret-security', variable: 'securityConfigFile')]) {
                    script {
                        sh 'cp -rf $securityConfigFile ./Backend/src/main/resources/application-security.yml'
                    }
                }
            }
        }

        stage("Build BE JAR to Docker Image") {
            steps {
                echo '백엔드 도커 이미지 빌드 시작!'
                dir("./Backend") {
                    // 빌드된 JAR 파일을 Docker 이미지로 빌드
                    sh "docker build -t shonee99/odi-be:latest ."
                }
                echo '백엔드 도커 이미지 빌드 완료!'
            }
        }

        stage("Push to Docker Hub-BE") {
            steps {

```



```

        echo '백엔드 도커 이미지를 Docker Hub에 푸시 시작!'
        withCredentials([usernamePassword(credentialsId: 'odi-docker-hub', usernameVariable: 'DOCKER_USERNAME',
            sh "docker login -u $DOCKER_USERNAME -p $DOCKER_PASSWORD"
        }) {
            dir("./Backend") {
                sh "docker push shonee99/odi-be:latest"
            }
            echo '백엔드 도커 이미지를 Docker Hub에 푸시 완료!'
        }
    }

stage("Deploy to EC2-BE") {
    steps {
        echo '백엔드 EC2에 배포 시작!'
        // 여기에서는 SSH 플러그인이나 SSH 스크립트를 사용하여 EC2로 연결하고 Docker 컨테이너 실행

        sh "docker rm -f backend"
        sh "docker rmi shonee99/odi-be:latest"
        sh "docker image prune -f"
        sh "docker pull shonee99/odi-be:latest && docker run -d -p 8080:8080 --name backend shonee99/odi-be:latest"

        echo '백엔드 EC2에 배포 완료!'
    }
}

stage("env download") {
    steps {
        withCredentials([file(credentialsId: 'secret-env-develop', variable: 'devConfigFile')]) {
            script {
                sh 'cp -rf $devConfigFile ./Frontend/.env.development'
            }
        }

        withCredentials([file(credentialsId: 'secret-env-prod', variable: 'prodConfigFile')]) {
            script {
                sh 'cp -rf $prodConfigFile ./Frontend/.env.production'
            }
        }
    }
}

stage("Build FE file to Docker Image") {
    steps {
        echo '프론트 도커 이미지 빌드 시작!'
        dir("./Frontend") {
            // 빌드된 파일을 Docker 이미지로 빌드
            sh "docker build -t shonee99/odi-fe:latest ."
        }
        echo '프론트 도커 이미지 빌드 완료!'
    }
}

stage("Push to Docker Hub-FE") {
    steps {
        echo '프론트 도커 이미지를 Docker Hub에 푸시 시작!'
        withCredentials([usernamePassword(credentialsId: 'odi-docker-hub', usernameVariable: 'DOCKER_USERNAME',
            sh "docker login -u $DOCKER_USERNAME -p $DOCKER_PASSWORD"
        }) {
            dir("./Frontend") {
                sh "docker push shonee99/odi-fe:latest"
            }
            echo '프론트 도커 이미지를 Docker Hub에 푸시 완료!'
        }
    }
}

stage('Deploy to EC2-FE') {
    steps {
        echo '프론트 EC2에 배포 시작!'

        sh "docker rm -f frontend"
        sh "docker rmi shonee99/odi-fe:latest"
        sh "docker image prune -f"
    }
}

```

```

        sh "docker pull shonee99/odi-fe:latest && docker run -d -p 3000:3000 --name frontend shonee99/odi-fe:lat

        echo '프론트 EC2에 배포 완료!'
    }
}

}

post {
    success {
        script {
            mattermostSend (
                color: 'good',
                icon: "https://jenkins.io/images/logos/jenkins/jenkins.png",
                message: "🎉 배포 성공 🎉"
            )
        }
    }
    failure {
        script {
            mattermostSend (
                color: 'danger',
                icon: "https://jenkins.io/images/logos/jenkins/jenkins.png",
                message: "실패라니 😞"
            )
        }
    }
}
}
}

```

3-8. ELK 스택 설정 - docker-compose.yml

```

version: '3'

services:
  elasticsearch:
    image: elasticsearch:8.6.2
    container_name: elasticsearch
    ports:
      - "9200:9200"
      - "9300:9300"
    environment:
      - "discovery.type=single-node"
      - "ES_JAVA_OPTS=-Xmx6g -Xms6g"
    mem_limit: 6g

  kibana:
    image: kibana:8.6.2
    container_name: kibana
    ports:
      - "5601:5601"
    links:
      - elasticsearch:elasticsearch
    mem_limit: 2g

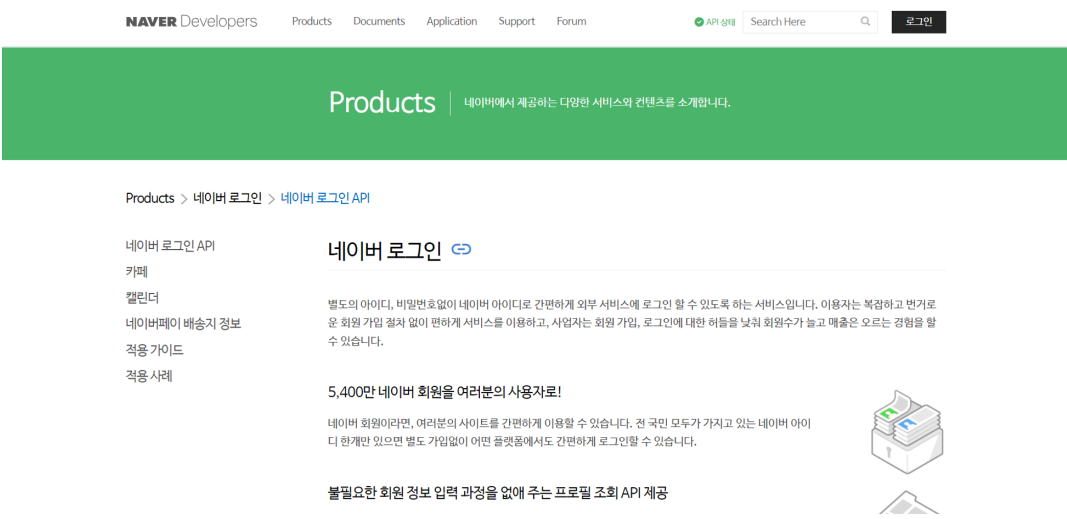
  logstash:
    image: logstash:8.6.2
    container_name: logstash
    volumes:
      - logstash:/usr/share/logstash
    ports:
      - "6000:6000"
    links:
      - elasticsearch:elasticsearch
    mem_limit: 4g

volumes:
  logstash:

```

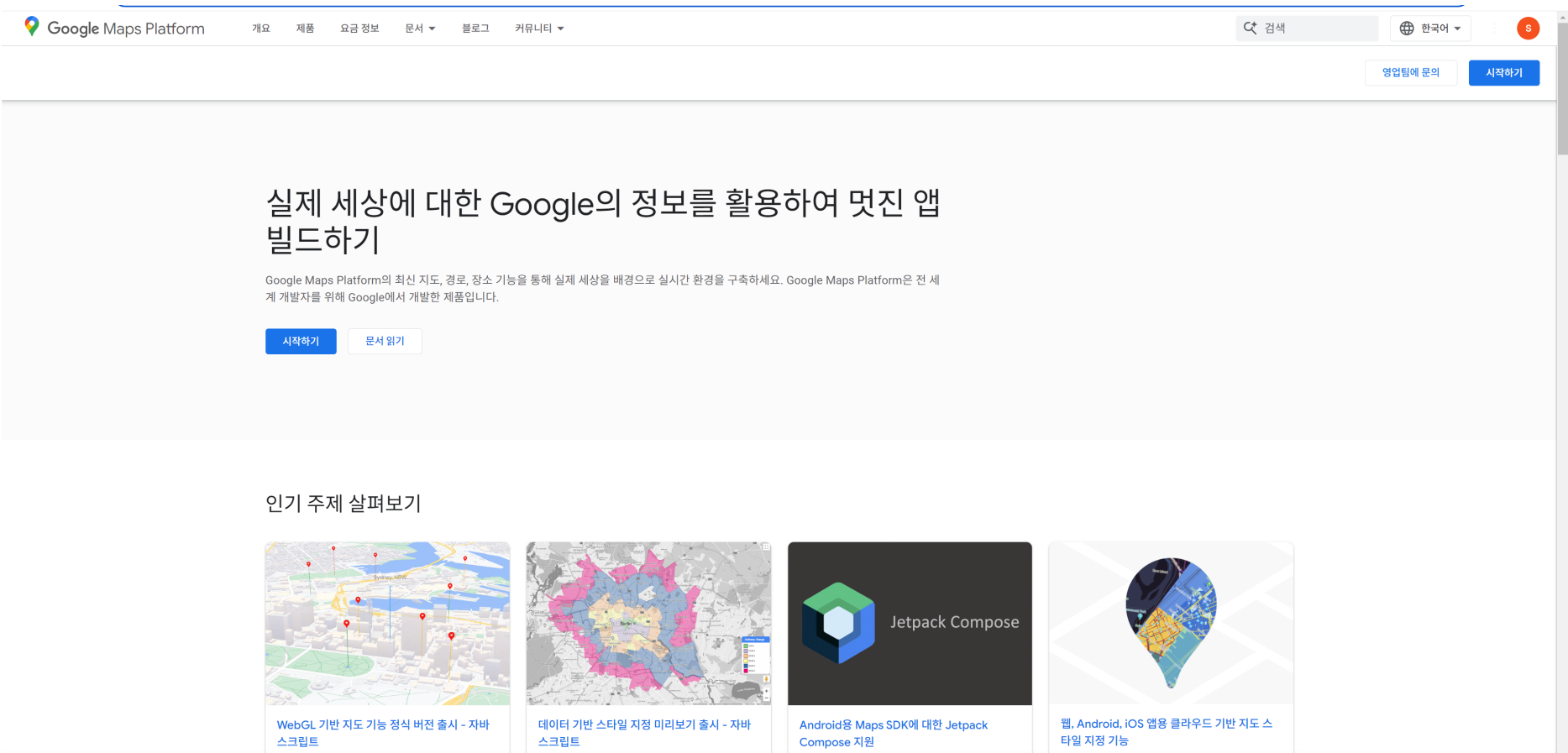

4. 외부 서비스 가입 및 활용에 필요한 정보

4-1. 네이버 로그인 API



<https://developers.naver.com/products/login/api/api.md>

4-2. 구글 지도 API



<https://developers.google.com/maps?hl=ko>

4-3. 네이버 지도 API

NAVER
CLOUD PLATFORM

소개*서비스*솔루션마켓플레이스*요금*고객지원*파트너가이드센터*

문약하기

문송

Application ServicesGeoLocationMapsCAPTCHAShortURLSimple & Easy Notification ServiceAPI GatewaySearch TrendNcloud

Maps

네이버 지도 기능을 활용하여 위치 기반 서비스를 만들 수 있는 API

이용 신청하기요금 계산하기

특징

상세 기능

적용 서비스

요금

사용 가이드

2023년 1월부터 일부 Maps API의 요금이 변경되었습니다.
요금 변경에 대한 더욱 자세한 내용은 공지사항 참고 부탁드립니다. (공지사항 바로가기)

국내 최고 수준의 지도 서비스를 제공하는 네이버 지도 API

네이버 지도를 활용하여 지도 관련 웹 서비스나 애플리케이션에 간단한 약도부터 유명 관광지 표시 등 다양한 정보를 표시할 수 있습니다.

높은 신뢰도

네이버 지도는 네이버와 지도 관련 전문 기관의 신뢰할 수 있는 정보로 구축한 서비스입니다. 또한 네이버 지도를 운영하며 축적한 노하우를 집약하여 지속적으로 발전시키고 개선하고 있습니다.

국내 1위의 지도 서비스

대한민국에서 가장 많은 사람들이 사용하는 국내 최대의 지도 서비스로 수시로 진행되는 자체 업데이트는 물론 사용자로부터 받은 수정사항을 실시간으로 반영하여 최신의 정확한 정보를 제공합니다.

위치 기반 서비스의 경쟁력 제고

물류, 판매, 통신, 유통 등 여러 사업 분야에서 활용할 수 있는 Maps, Directions, Geocoding 관련 다양한 API 기능을 제공하여 위치와 이동 관련 정보가 중요한 비즈니스를 운영하는 사업자들에게 높은 서비스 경쟁력을 제공합니다.

최신 POI 데이터 보유

네이버 지도 검색을 사용하는 사용자 수와 클릭 수에 근거하여 신뢰도 높은 평점 모델을 반영한 관심 지점(Point of Interest, POI)의 최신 정보를 제공합니다.

https://www.ncloud.com/product/applicationService/maps

4-4. 토스페이먼츠 API

payments 개발자센터가이드API & SDK샌드박스커뮤니티·지원블로그

검색 Ctrl + K로그인

API

시작하기

코어 API

결제

자동결제(빌링)

거래

정산

현금영수증

지급대행

프로모션

브랜드페이 API

API 테스트

SDK

결제위젯

JavaScript

Android

IOS

React Native

Flutter

브랜드페이

API 버전 정책

에러 코드

기관 코드

데이터 타입

FAQ

API & SDK / 코어 API

Version 2022-11-16

코어 API

토스페이먼츠 API 엔드포인트(Endpoint)와 객체 정보, 파라미터, 요청 및 응답 예제를 살펴보세요.

API 키인증 및 기타 헤더 설정요청·응답 본문보안

API를 사용하기 전에 버전 정책을 확인하세요. API 응답이나 웹hook 이벤트 본문에 새로운 필드가 추가와 같이 하위 호환을 지원하는 변경 사항은 버전 릴리즈 없이 기존 API에 반영됩니다. 유의하세요. 자세한 내용은 요청·응답 본문을 참고하세요.

결제

결제에 관한 모든 API와 결제 API의 응답으로 돌아오는 Payment 객체, Cancel 객체를 살펴봅니다.

Payment 객체

결제 정보를 담고 있는 객체입니다. 결제 한 건의 결제 상태, 결제 취소 기록, 매출 전표, 현금영수증 정보 등을 자세히 알 수 있습니다. 결제가 승인됐을 때 응답은 Payment 객체로 항상 동일합니다. 객체의 구성은 결제수단(카드, 가상계좌, 간편결제 등)에 따라 조금씩 달라집니다.

객체 상세

version string v12Payment 객체의 응답 버전입니다. 버전 2022-06-08부터 날짜 기반 버저닝을 사용합니다.

목차

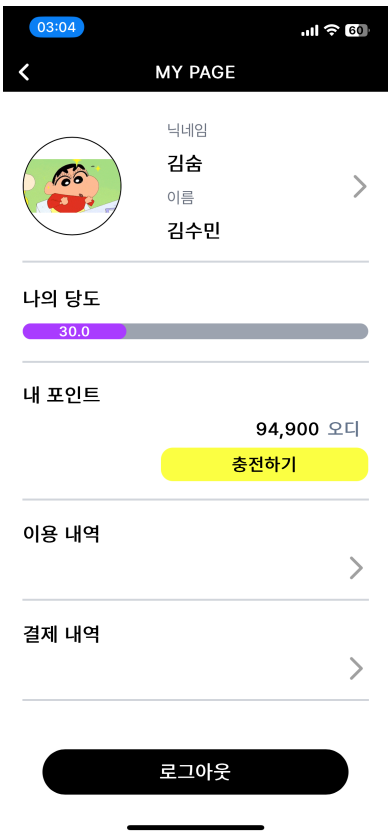
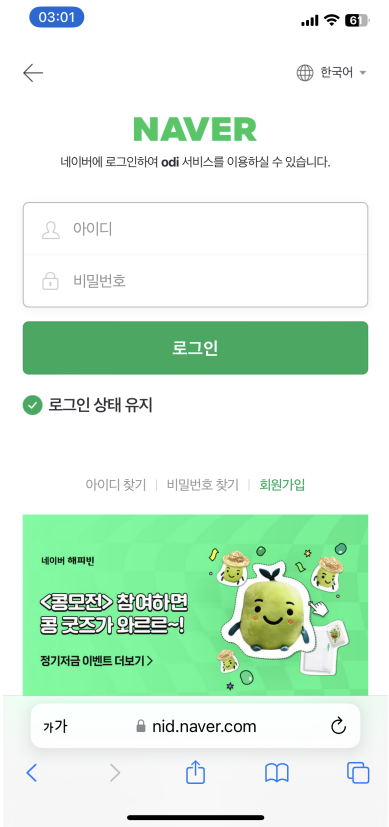
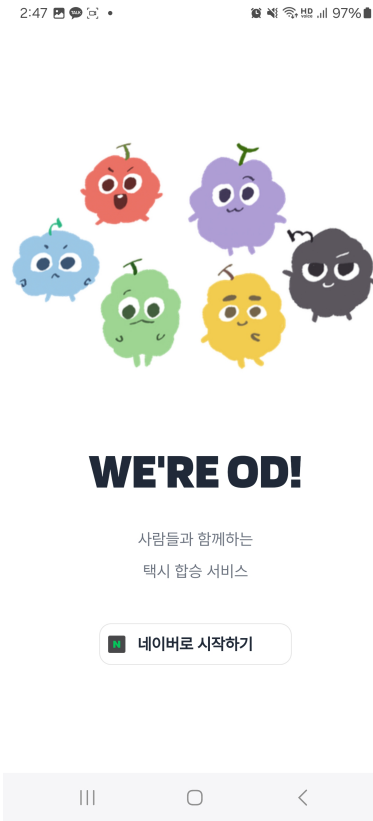
결제Payment 객체결제 승인paymentKey로 결제 조회orderId로 결제 조회결제 취소가상계좌 발급 요청자동결제Billing 객체customerKey로 카드 발령키 발급authKey로 카드 발령키 발급카드 자동결제 승인거래Transaction 객체거래 조회정산Settlement 객체정산 조회수동 정산 요청현금영수증CashReceipt 객체현금영수증 발급 요청현금영수증 발급 취소 요청현금영수증 조회지급대행Submail 객체서비스 등록서비스를 조회서비스를 수정서비스를 삭제Payout 객체

https://docs.tosspayments.com/reference

5. 시연 시나리오

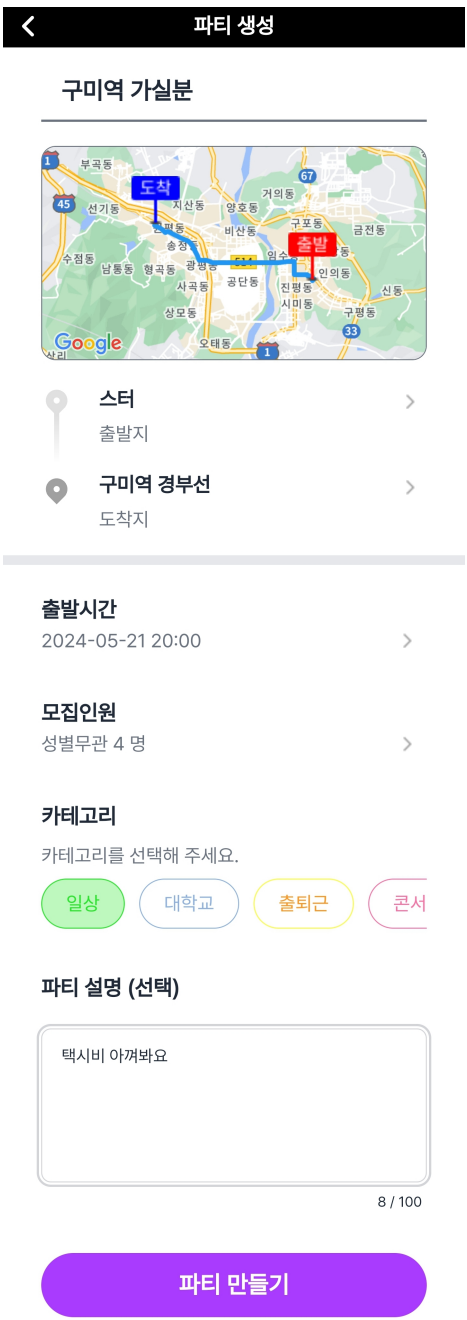
5-1. 로그인

- 네이버 간편 소셜 로그인 제공



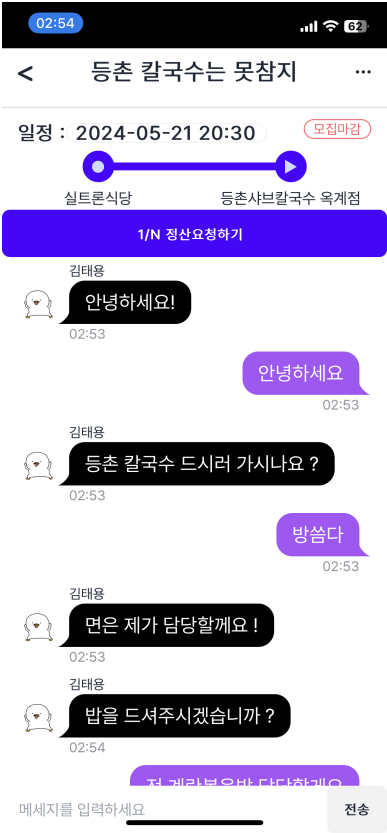
5-2. 동승 구인 글 작성 (파티 생성)

- 출발지와 목적지 장소 선택(장소 자동완성 및 마커 이동 이용)
- 출발지, 목적지, 시간, 인원, 카테고리 정보를 포함하여 동승 구인 글 작성



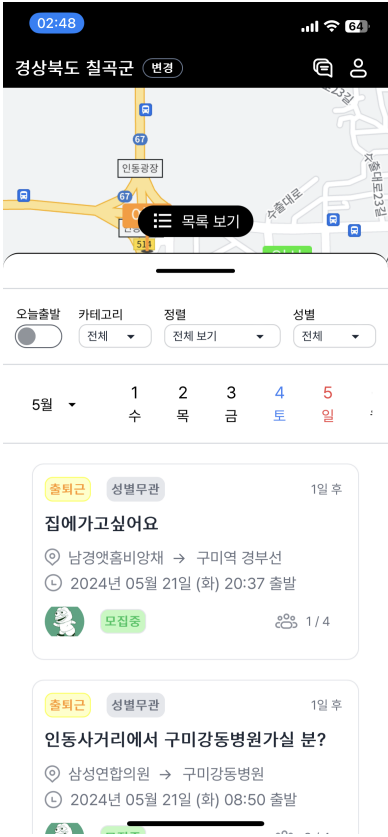
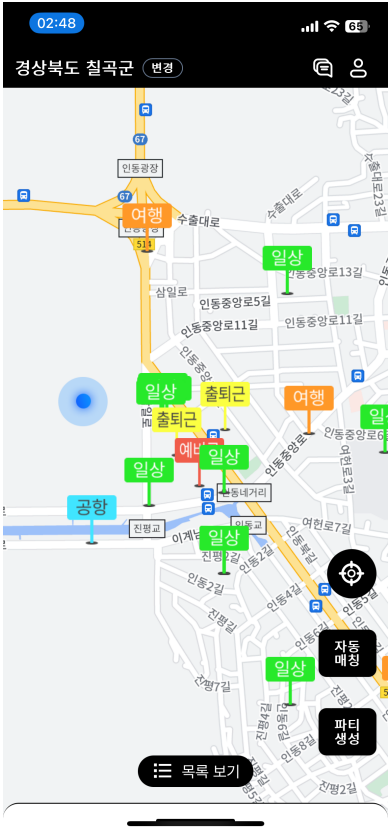
5-3. 채팅

- 실시간 채팅 기능



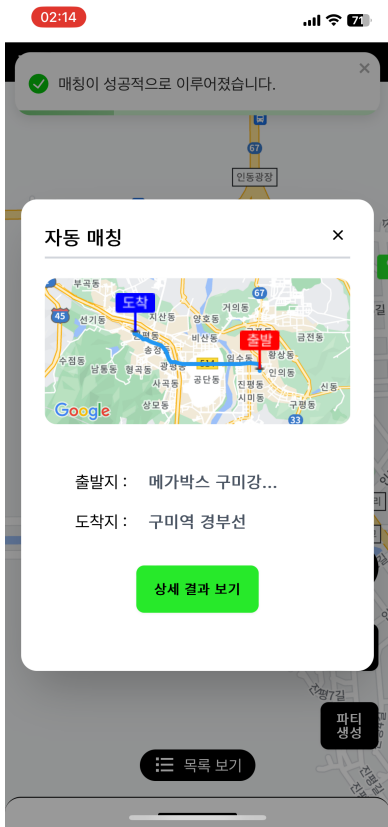
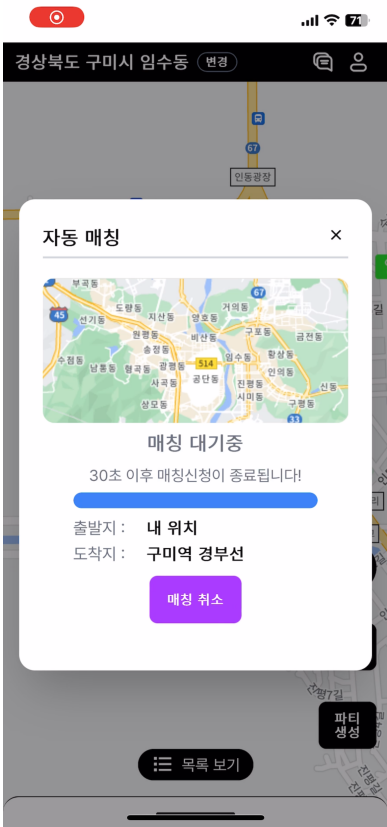
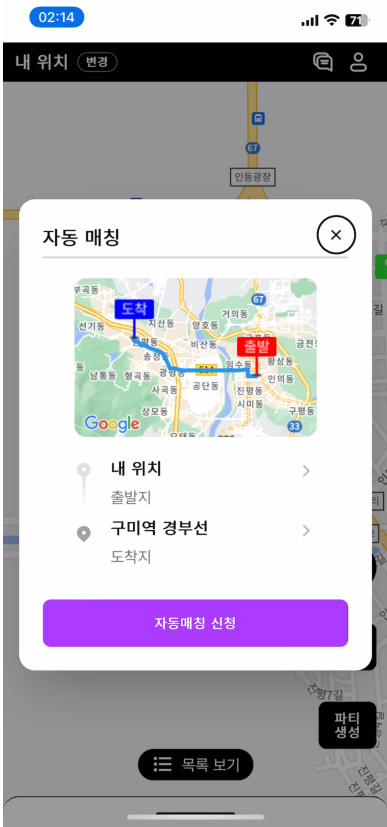
5-4. 파티 목록 조회

- 거리순, 출발시간 가까운 순 정렬 기능
- 출발일자, 카테고리, 성별 필터 기능



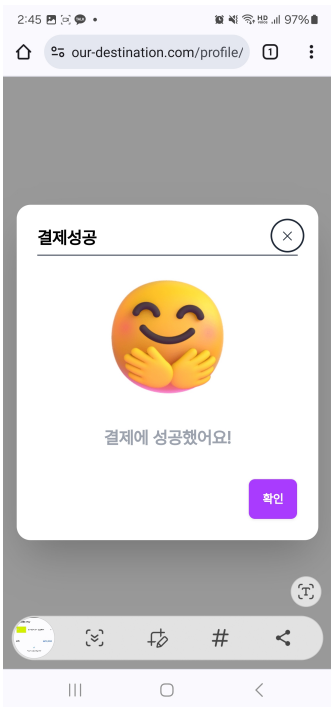
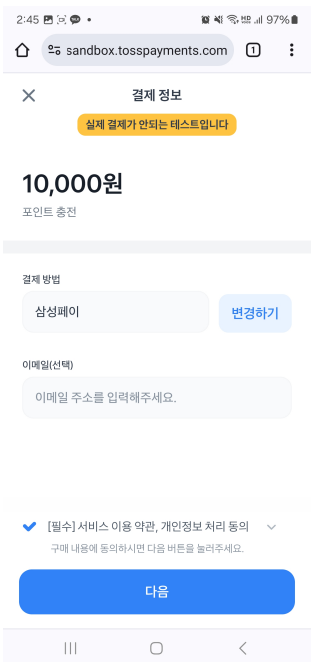
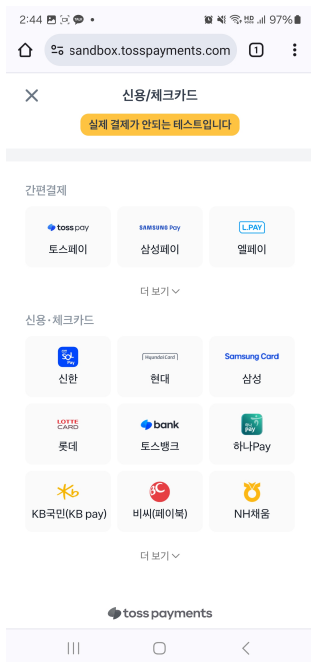
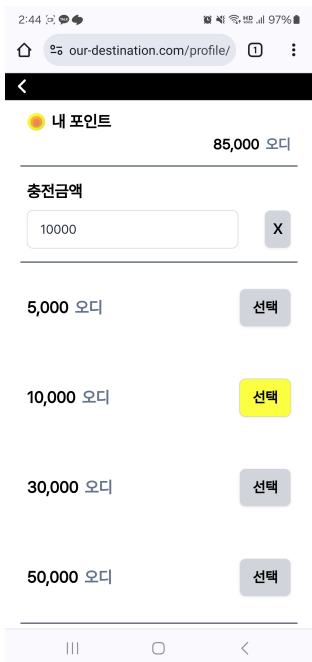
5-5. 자동 매칭

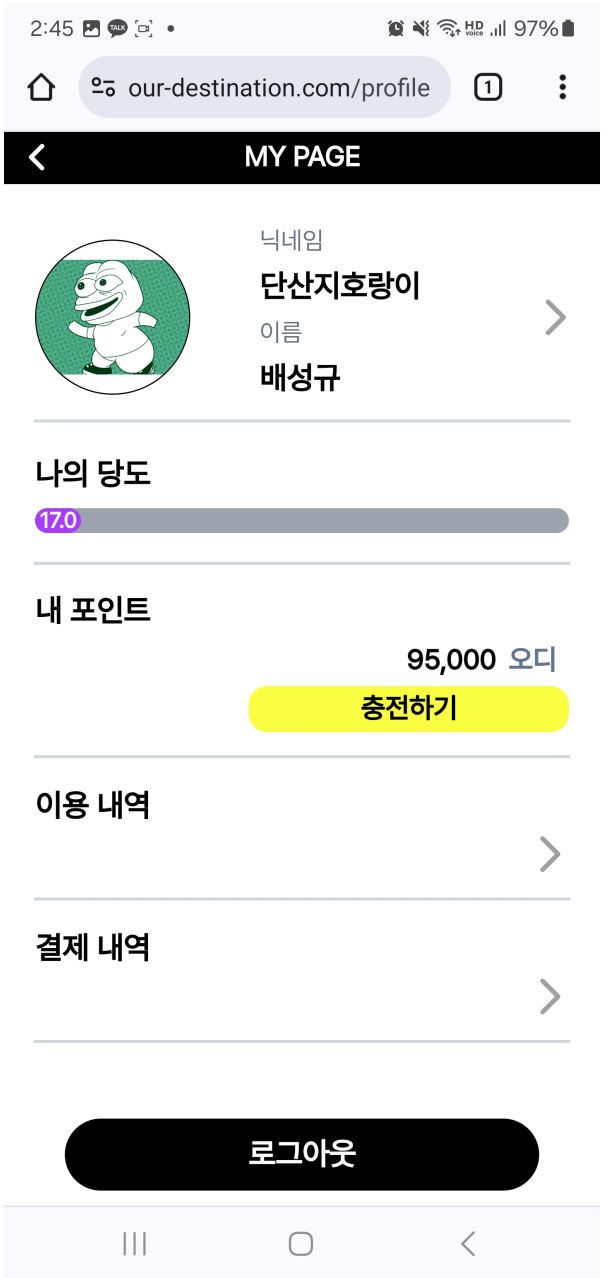
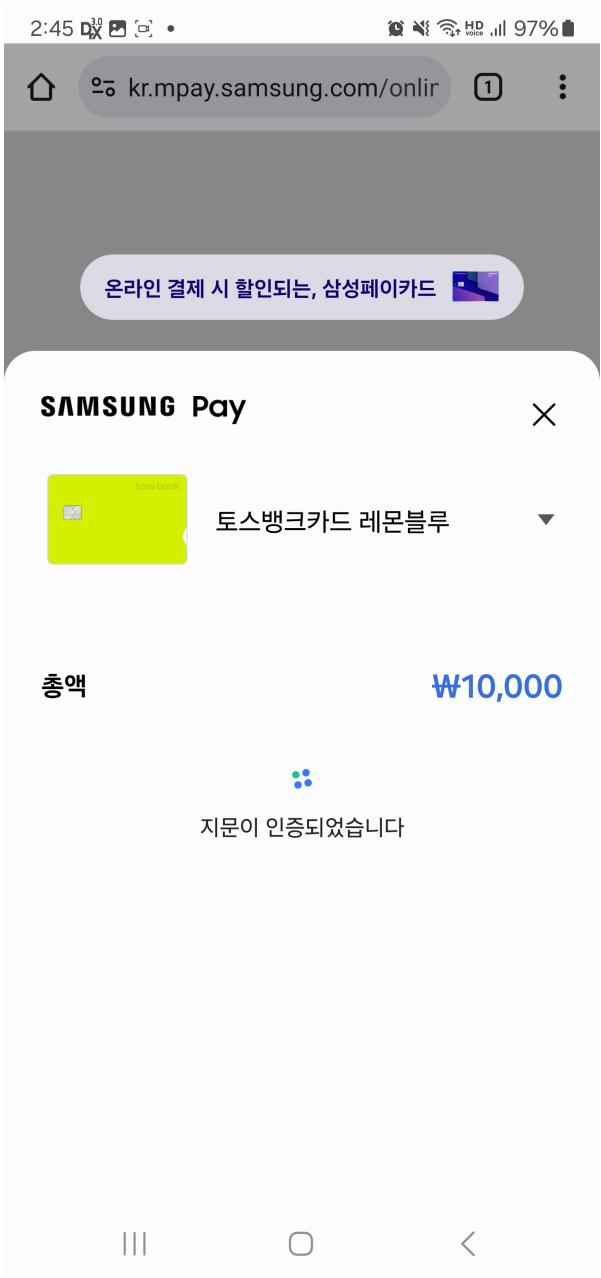
- 실시간 자동 매칭 제공



5-6. 포인트 충전

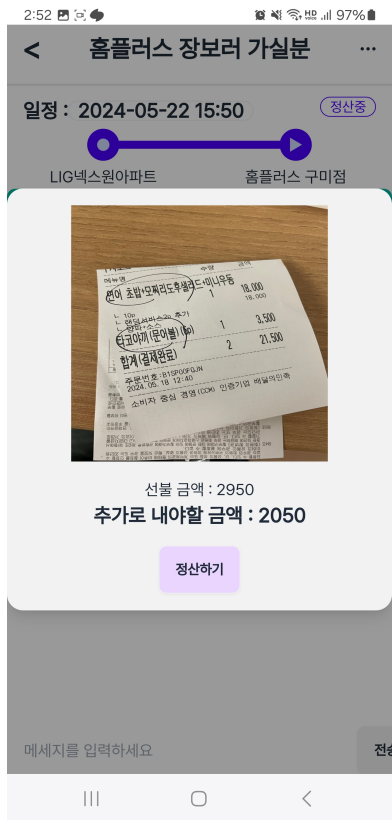
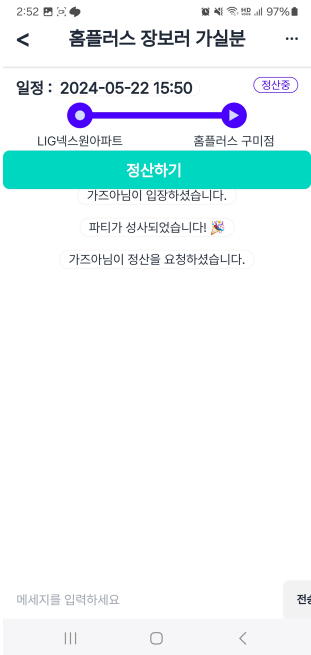
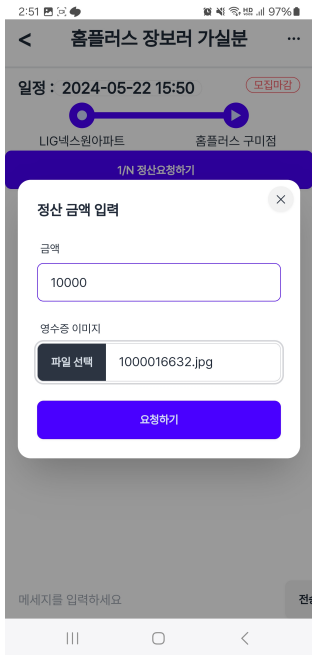
- Tosspayments 결제 API를 이용하여 파티 정산에 사용하는 포인트 충전





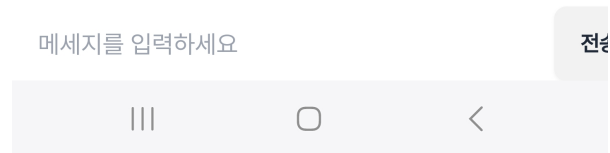
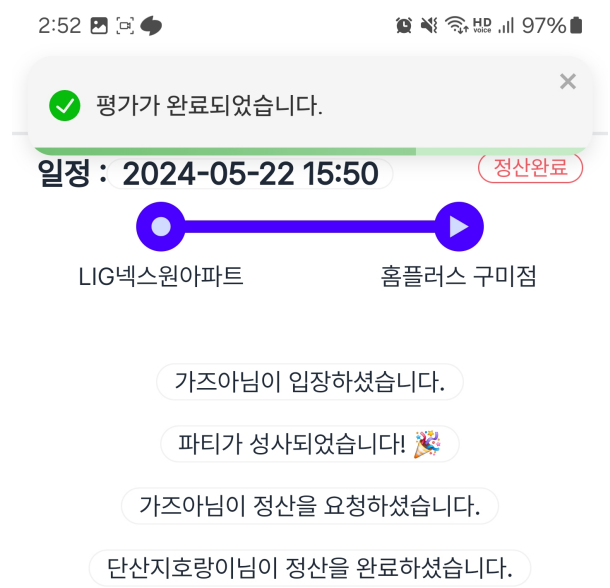
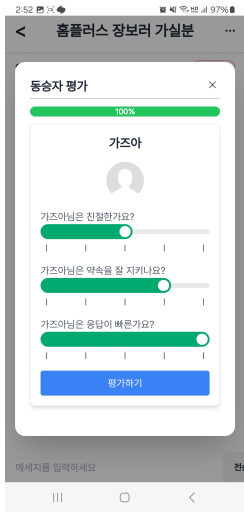
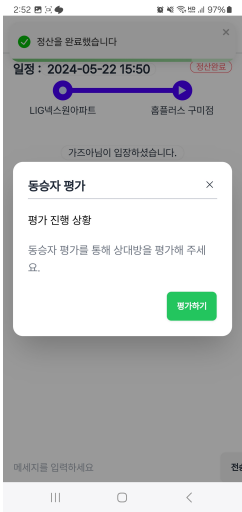
5-7. 정산하기

- 파티 성사 시 참여자 대상으로 선불 금액 차감
- 택시 이용 후 결제자가 영수증과 함께 정산 요청
- 결제자 제외한 참여자가 영수증 확인 후 정산



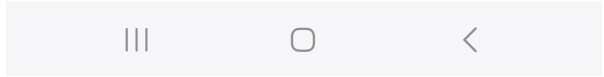
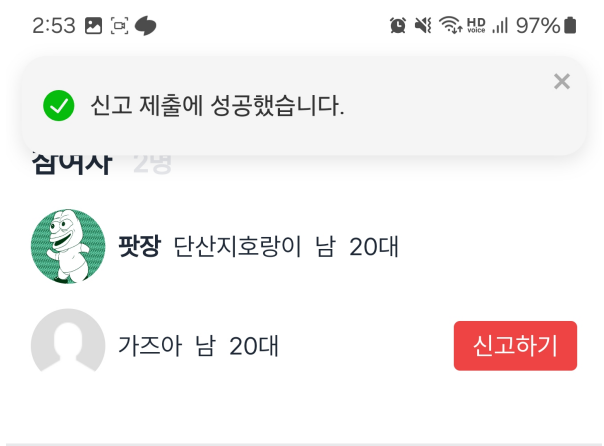
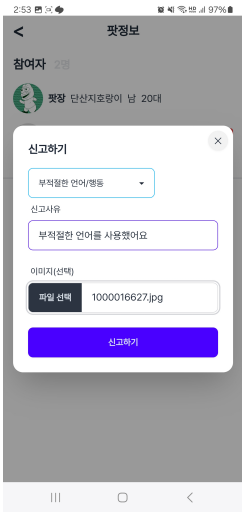
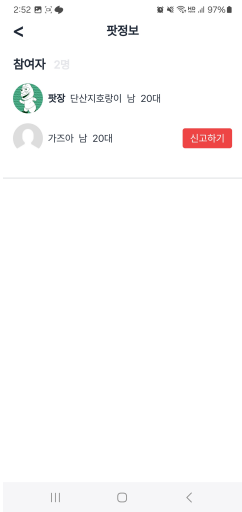
5-8. 동승자 평가 : 오디 당도

- 정산이 끝난 후 함께 택시를 이용한 동승자들에 대한 평가 진행



5-9. 신고하기

- 미정산 등 파티에서 비매너 행위를 한 유저에 대해 신고하기 기능



5-10. 파티 신청

- 매칭을 신청하면 팟장이 수락 또는 거절
- 신청이 수락되면 파티원이 되고 팟 채팅에 참여 가능

